

Preliminary Package Exploration

Li Shandross

10/9/2021

```
## Error in assign(generic, get(generic, asNamespace("greybox")), envir = asNamespace(pkg)) :  
## cannot change value of locked binding for 'forecast'
```

We explore the usage of the THieF, MAPA, hts, and fable R packages to a select amount of Covid-19 data in this report. More specifically, we summarize the functionality of each package and their usefulness to building the hierarchical time series forecasting model, paying particular attention to their application to probabilistic forecasts.

THieF

Description of Package

A package for Temporal Hierarchical Forecasting. It only contains three functions: `tsaggregates`, `reconcilethief`, and `thief`.

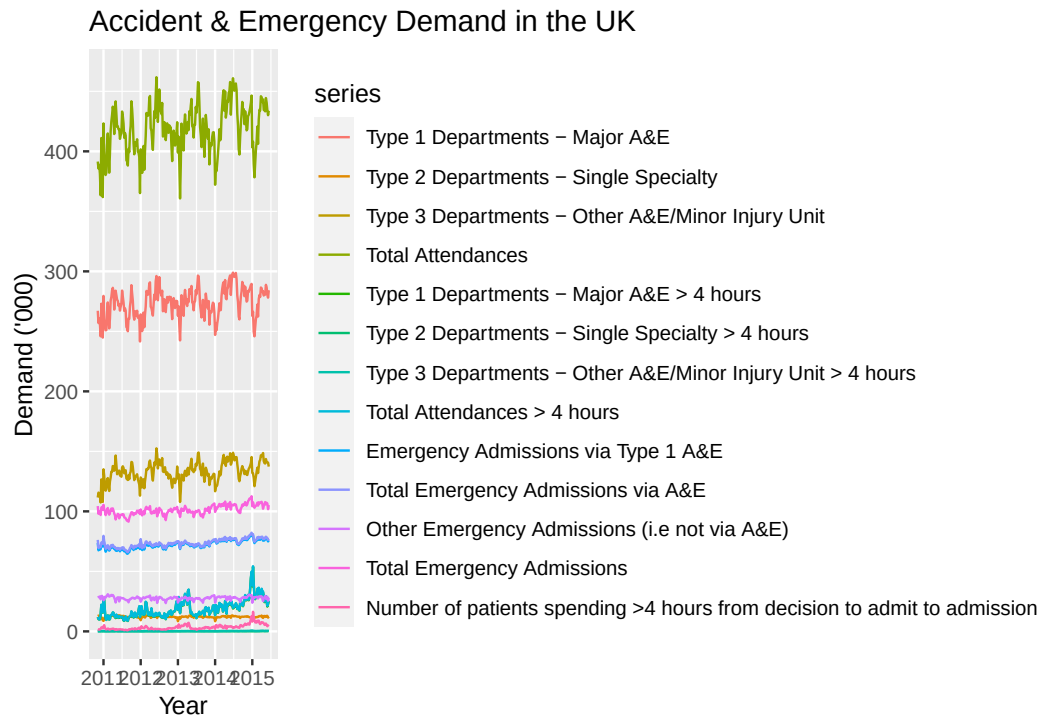
- `tsaggregates` takes in a time series object then performs aggregation of the data that outputs a list of vectors for each level aggregation. The levels of aggregation depend on the time unit of original data, the smallest being the given time unit, then double that unit, quadruple that unit, quarterly, biannual, and finally annual. If you are interested in changing the default aggregation levels, you must input a list object to specify your desired aggregation levels. The results from `tsaggregates` may be plotted using various applicable functions, e.g. `plot()` or `autoplot()`.
- `reconcilethief` reconciles the input list object (of base forecasts) to create new reconciled forecasts using the sample covariance estimator Λ , likely the series variance scaling estimator Λ_V , described in *Forecasting with Temporal Hierarchies*. The resulting reconciled forecasts are given in a list object that contains both point forecasts and two confidence intervals (80 and 95) of those point forecasts. The point forecasts are then typically then plotted alongside the base forecasts to roughly compare their accuracy by eye. Note that this function throws an error if there is not enough data in the original time series to aggregate to a full data point for the highest aggregation level.
- `thief` is an overarching function that performs `tsaggregates` and `reconcilethief` (calculating base forecasts in between) in one big wrapper function. `thief` takes a time series object as input, computes temporal aggregates, calculates base forecasts, uses the base forecasts to create reconciled forecasts using the specified method, then finally returns reconciled forecasts. Note that `thief` only outputs point forecasts, discarding much of the output `reconcilethief`'s output includes.

THieF does not include any functions to calculate base forecasts. Instead, the examples below use the function `forecast` from the Forecast package to generate base forecasts, selecting an automatic ARIMA model. The package Forecast has been retired since the creation of THieF and Hyndman et. al's paper *Forecasting with Temporal Hierarchies* was published; now it is suggested that package Fable be used instead. Fable will be explored later in this write up.

Demonstration and Analysis

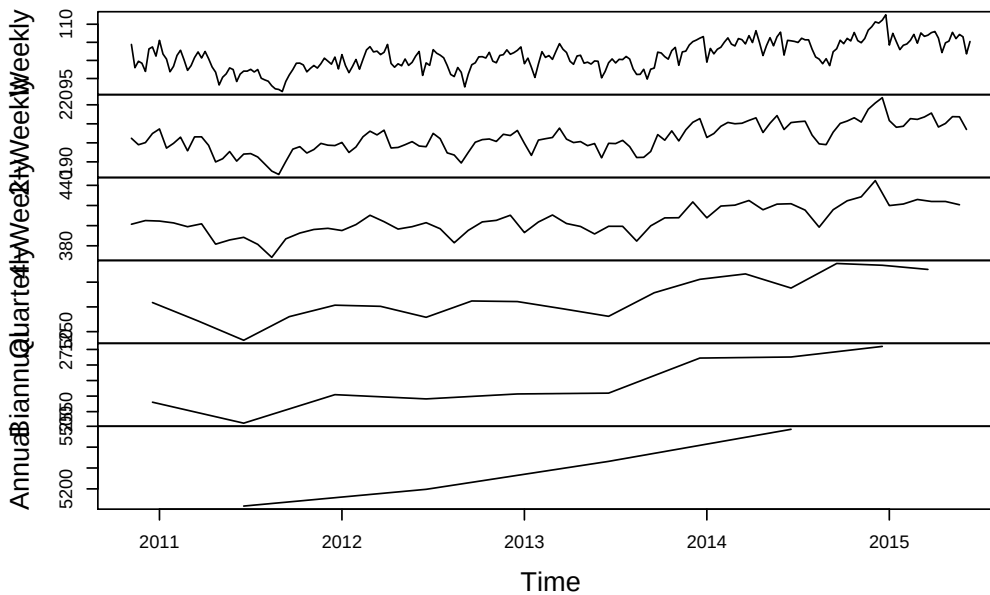
```
## Example1 - AEdemand
library(ggplot2)

autoplot(AEdemand, xlab="Year", ylab="Demand ('000)") +
  ggtitle("Accident & Emergency Demand in the UK")
```



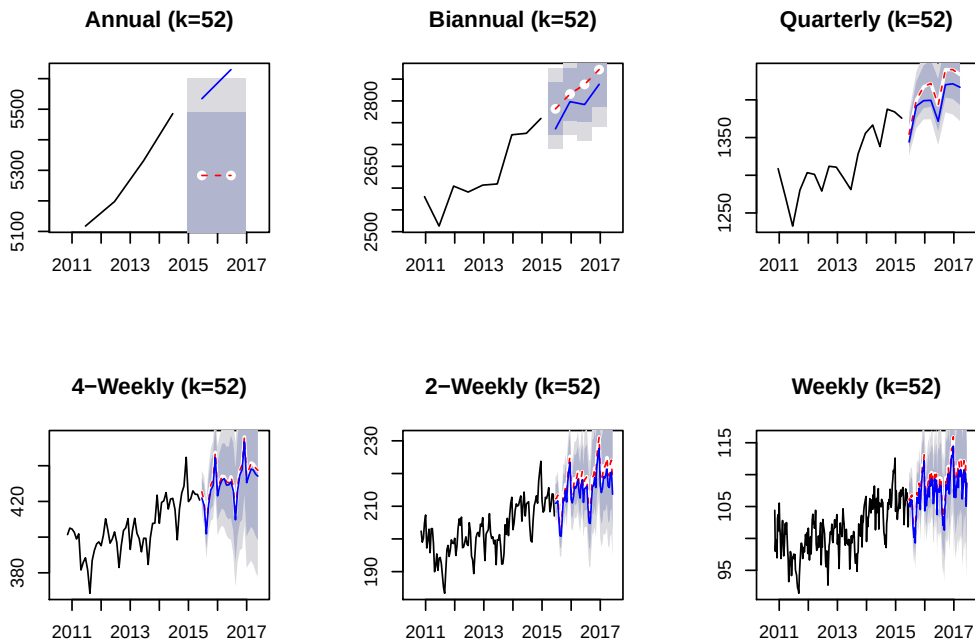
```
# Demonstration of the adjustment of all temporal aggregates
# using Total Emergency Admissions (pink)
total <- AEdemand[,12]
totalagg <- tsaggregates(total) #tsaggregate()
plot(totalagg, main="Total Emergency Admissions")
```

Total Emergency Admissions



```
# Base forecasts
base <- list()
for(i in 1:5)
  base[[i]] <- forecast(auto.arima(totalagg[[i]])) # creates upper and lower bounds
base[[6]] <- forecast(auto.arima(totalagg[[6]]), h=2)

# Reconciled forecasts
reconciled <- reconcilethief(base) #reconcilethief() # retains upper and lower bounds
main <- paste(names(totalagg), " (k=",
              52/unlist(lapply(reconciled,frequency)),")",sep="")
par(mfrow=c(2,3))
for(i in 6:1)
{
  ylim <- range(base[[i]]$mean, base[[i]]$x, reconciled[[i]]$mean)
  plot(base[[i]], main=main[i], fcol='white',
        plot.conf=FALSE, ylim=ylim, xlim=c(2010.5,2017.5))
  #polygon(c(2015.45, 2020, 2020, 2015.45),
  #        #c(0, 0, 1e5, 1e5), col='grey', border=FALSE)
  lines(base[[i]]$mean, col='red', lty=2)
  lines(reconciled[[i]]$mean, col='blue')
}
```

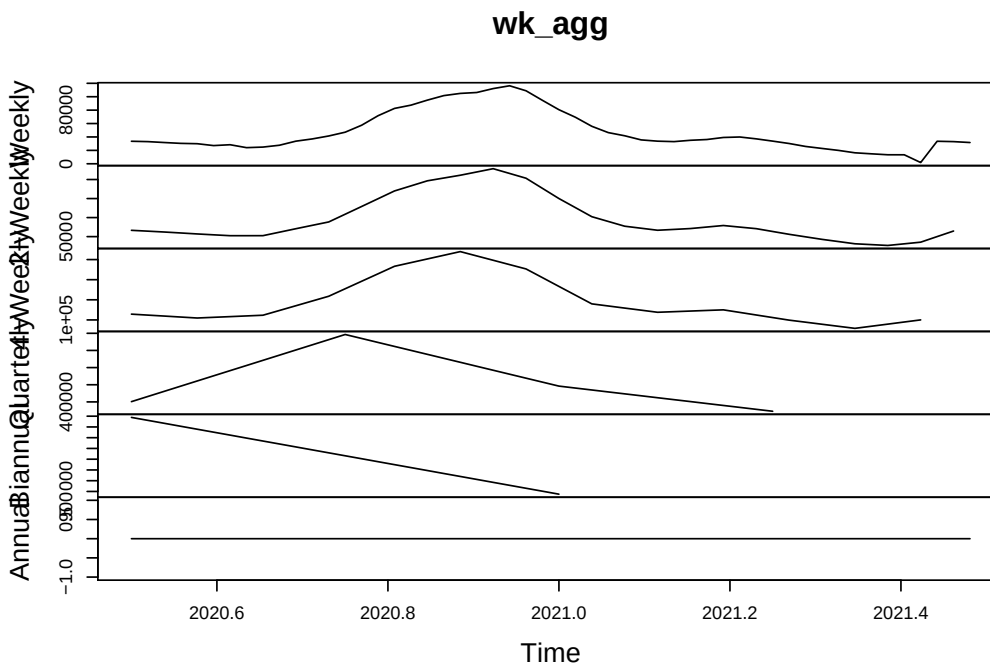


```
# load in Covid-19 Hosp Truth Data
hosp_truth <- load_truth("HealthData",
  "inc hosp",
  temporal_resolution="weekly",
  data_location = "remote_hub_repo") %>%
  dplyr::filter(target_end_date >= as.Date("2020-06-28"),
    target_end_date <= as.Date("2021-06-27"),
    location == "US") %>%
  mutate(day = wday(target_end_date), epi_week = epiweek(target_end_date)) %>%
  dplyr::select(value, epi_week, day)

# hosp truth data by day
ht_day <- hosp_truth$value
ht_day_ts <- ts(ht_day, start = c(2020, 179), end = c(2021, 178), frequency = 365)

# hosp truth data by week
ht_wk <- hosp_truth %>%
  group_by(epi_week) %>%
  mutate(wk_total = sum(value)) %>%
  ungroup() %>%
  distinct(epi_week, .keep_all=TRUE)
ht_wk_ts <- ts(ht_wk$wk_total, start = c(2020, 27), end = c(2021, 26), frequency = 52)

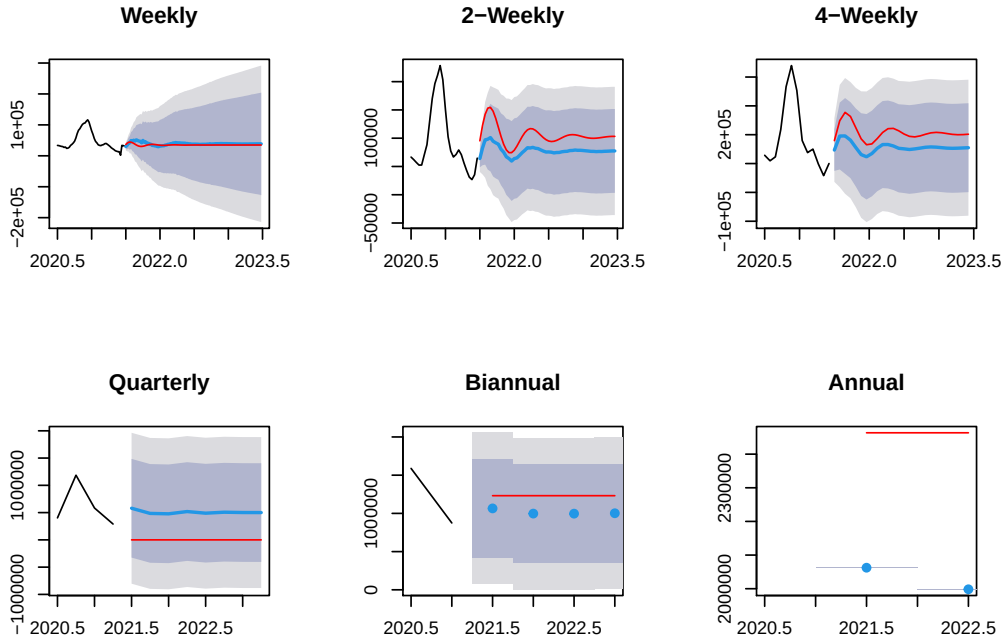
# Construct aggregates (from weekly data)
wk_agg <- tsaggregates(ht_wk_ts)
plot(wk_agg)
```



```
# Compute base forecasts
base_fc <- list()
for(i in seq_along(wk_agg))
  base_fc[[i]] <- forecast(auto.arima(wk_agg[[i]]), h=2*frequency(wk_agg[[i]]))
# creates upper + lower bounds

# Reconcile forecasts
reconciled_fc <- reconcilethief(base_fc)

# Plot forecasts before and after reconciliation
par(mfrow=c(2,3))
for(i in seq_along(base_fc))
{
  plot(reconciled_fc[[i]], main=names(wk_agg)[i]) # plots bounds
  lines(base_fc[[i]]$mean, col='red') # plots red mean line
}
```



Concluding Remarks on Usefulness

THieF seems like it will provide an important backbone for applying Temporal Hierarchies to Covid-19 data. However, as highlighted by the short example using Covid-19 hospitalization truth data, there are a couple of aspects that may need to be adjusted.

First, the applicable functions in THieF seem to require at least a full year's worth of data to work properly (without throwing any errors). However, the final forecasts only predicts for shortly before the final date of truth data fed-in. It would be helpful to see if forecasts could start from earlier. Another potential solution would be to make the highest level of aggregation smaller, perhaps 4-monthly or quarterly since we have far less Covid-19 data than other examples of time series.

Second, it would be nice to be able to adjust the quantiles produced for the forecasts to be 50 and 95 instead of 80 and 95 to match those used in visualizations by the COVID-19 Forecast Hub. Potentially, we might want all of the quantiles specified by the Forecast Hub to be able to more fully compare a Temporal Hierarchy model to the other models currently submitting to the Hub for incident hospitalizations. This would likely be part of exploration for the Forecast and Fable packages later.

MAPA

Based on the description of MAPA (Multiple Aggregation Prediction Algorithm) in *Forecasting with Temporal Hierarchies* and a quick scan of the paper introducing MAPA, *Improving Forecasting By Estimating Time Series Structural Components Across Multiple Frequencies*, it seems that MAPA may be outside of the scope of my thesis work, or it should at least not be considered until further into the process when I already have solidly implemented the THieF methods.

Forecast

The package `Forecast` has been retired since the creation of `THieF` and publication of Hyndman et. al's paper *Forecasting with Temporal Hierarchies*. It is now suggested that package `Fable` be used instead. `Fable` will be explored later in this write up.

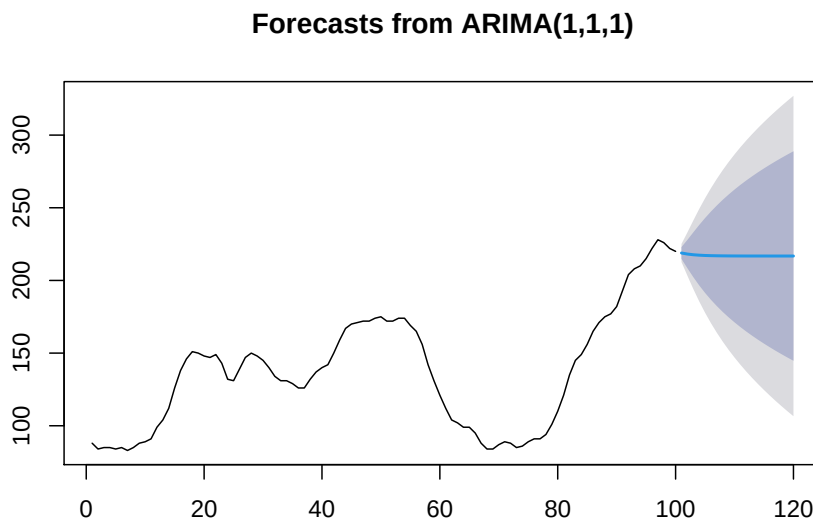
Description of Package

`THieF` automatically loads `Forecast`, likely due its dependencies on `Forecast` functions. More specifically, `THieF` uses `auto.arima` and `forecast`, the former of the two which takes in a univariate time series object, unlike that of the `Arima` in `Forecast` and `ARIMA` in `Fable`.

- `auto.arima` takes in a univariate time series object and determines the best-fitting ARIMA model according to either AIC, AICc or BIC value for the time series data. “The function conducts a search over possible model within the order constraints provided.”
- `forecast` “takes in a time series or time series model for which forecasts are required” and creates forecasts. The particular methods invoked are dependent on the class of the the time series or time series model object fed in. Here, h is defined to be the number of periods for forecasting (h likely cannot take on a character value). When used for the `THieF` method, the time series model used is the best arima model (found using `auto.arima`) and h is specified to be two times the highest aggregation level.

Demonstration and Analysis

```
fit <- auto.arima(WWWusage)
forecasts <- forecast(fit, h=20)
plot(forecasts)
```



Concluding Remarks on Usefulness

Since THieF automatically loads Forecast, it may be worth simply continuing to use the functions in Forecast instead of trying to switch over to the corresponding ones in Fable (if they exist), especially because there does not appear to be an equivalent to `auto.arima` in Fable (or Fabletools). THieF’s dependency on the functionality of `auto.arima` and `forecast` may prove to be an obstacle to switching over to Fable entirely. Further, this is only a few functions and Forecast’s official retirement occurred less than six months ago, meaning it is not largely out of date.

hts

The package `hts` has been retired since the creation of THieF and publication of Hyndman et. al’s paper *Forecasting with Temporal Hierarchies*. It is now suggested that package Fable be used instead. Fable will be explored later in this write up. (Note that `hts` was retired around the same time as Forecast.)

`hts` was written prior to *Forecasting with Temporal Hierarchies* and the initial write-up included a slightly different equation for obtaining reconciled forecasts. Based on the documentation of several relevant functions, it appears that the equations used to calculate reconciled forecasts in the package are not the same as the ones in *Forecasting with Temporal Hierarchies*. Nevertheless, this package may provide some useful functions to calculate components used in the THieF method, such as the `smatrix` for the summing matrix S . It also includes an aggregation function `aggrts` and a function `combinef` to obtain forecasts using the optimal combination method (the method used in THieF to obtain reconciled forecasts, though the equation used differs slightly).

Fable

Description of Package

Since the description of `hts` and Forecast direct users to Fable instead, it seems that Fable may be a combination of those two packages, potentially with additional functionality.

Fable automatically loads Fabletools, a package that “provides additional tools, helpers, and data structures for developing models and time series functions for Fable and extension packages.”

A `forecast` function exists in Fabletools, which may be identical (or similar enough) to the one in Forecast so that if we use a different method than `auto.arima` for the reconciled forecasts (or choose to manually find the best model), it may be possible to use the functions in Fable. On the other hand, it is unclear if any of the noted functions of use in `hts` made it into Fable, so we likely cannot use Fable as a replacement for `hts` for our purposes.

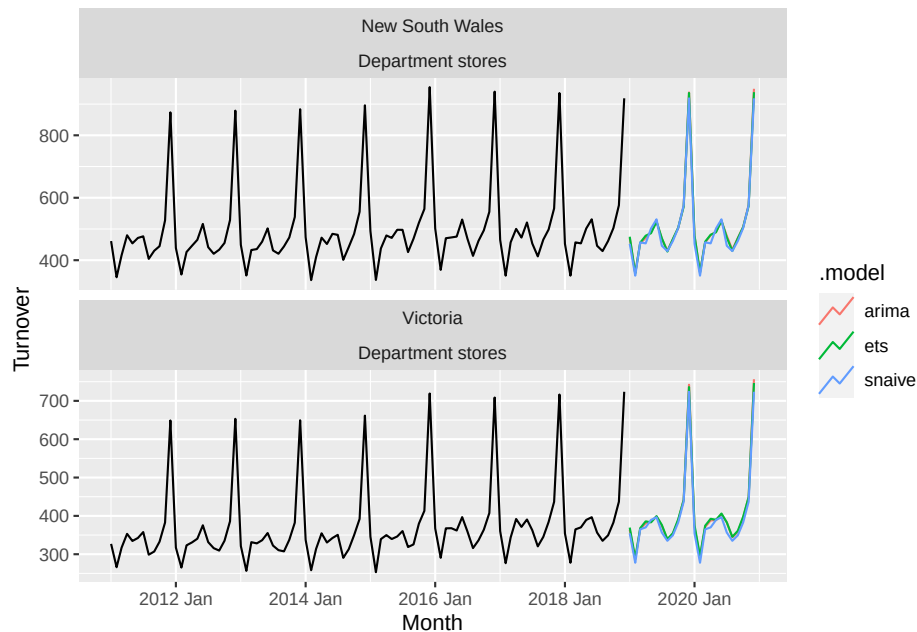
It is unclear whether it would be better to migrate to Fable in favor of Forecast given THieF’s dependency on, yet limited usage of, functions from the Forecast package. Fable may provide additional, more up-to-date functions that could be used, but the existing, older functions in Forecast may suffice as is.

Depending on the difficulty and/or time required, it may be helpful to test if Fable’s new versions of functions useful for the THieF methodology yield the same results as the older ones in Forecast.

- `forecast` takes in a time series model and produces future predictions. “If the response variable has been transformed in the model formula, the transformation will be automatically back-transformed (and bias adjusted if `bias_adjust` is TRUE).” Here, h is defined to be the forecast horizon, and it can take in character strings such as “2 years”.

Demonstration and Analysis

```
aus_retail %>%
  filter(
    State %in% c("New South Wales", "Victoria"),
    Industry == "Department stores"
  ) %>%
  model(
    ets = ETS(box_cox(Turnover, 0.3)),
    arima = ARIMA(log(Turnover)),
    snaive = SNAIVE(Turnover)
  ) %>%
  forecast(h = "2 years") %>%
  autoplot(filter(aus_retail, year(Month) > 2010), level = NULL)
```



```
if (requireNamespace("fable", quietly = TRUE)) {
  library(fable)
  library(tsibble)
  library(tsibbledata)
  library(dplyr)
  library(tidyr)

  # Forecasting with an ETS(M,Ad,A) model to Australian beer production
  beer_fc <- aus_production %>%
    model(ets = ETS(log(Beer) ~ error("M") + trend("Ad") + season("A"))) %>%
    forecast(h = "3 years")

  # Compute 80% and 95% forecast intervals
  beer_fc %>%
    hilo(level = c(80, 95))
}
```

```

beer_fc %>%
  autoplot(aus_production)

# Forecasting with a seasonal naive and linear model to the monthly
# "Food retailing" turnover for each Australian state/territory.
library(dplyr)
aus_retail %>%
  filter(Industry == "Food retailing") %>%
  model(
    snaive = SNAIVE(Turnover),
    ets = TSLM(log(Turnover) ~ trend() + season()),
  ) %>%
  forecast(h = "2 years 6 months") %>%
  autoplot(filter(aus_retail, Month >= yearmonth("2000 Jan")), level = 90)

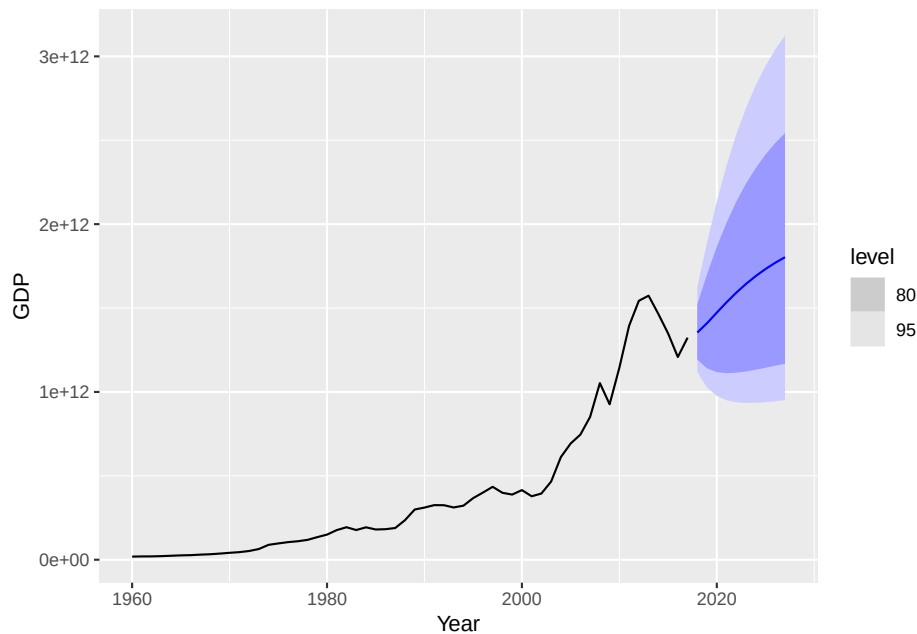
# Forecast GDP with a dynamic regression model on log(GDP) using population and
# an automatically chosen ARIMA error structure. Assume that population is fixed
# in the future.
aus_economy <- global_economy %>%
  filter(Country == "Australia")

fit <- aus_economy %>%
  model(lm = ARIMA(log(GDP) ~ Population))

future_aus <- new_data(aus_economy, n = 10) %>%
  mutate(Population = last(aus_economy$Population))

fit %>%
  forecast(new_data = future_aus) %>%
  autoplot(aus_economy)
}

```



Concluding Remarks on Usefulness

Fable seems like it could contain functions that are useful, but it would require figuring out how to recreate the output of `auto.arima` from Forecast and looking further into the package to check if any useful functions from hts actually did migrate over. While these are doable tasks, they requires extra work, which may not be worth it depending on how difficult and/or time consuming it is. Beyond some initial exploration to determine the difficulty and time required to do so, it may be worth simply tabling Fable, despite it being the successor of Forecast and hts.